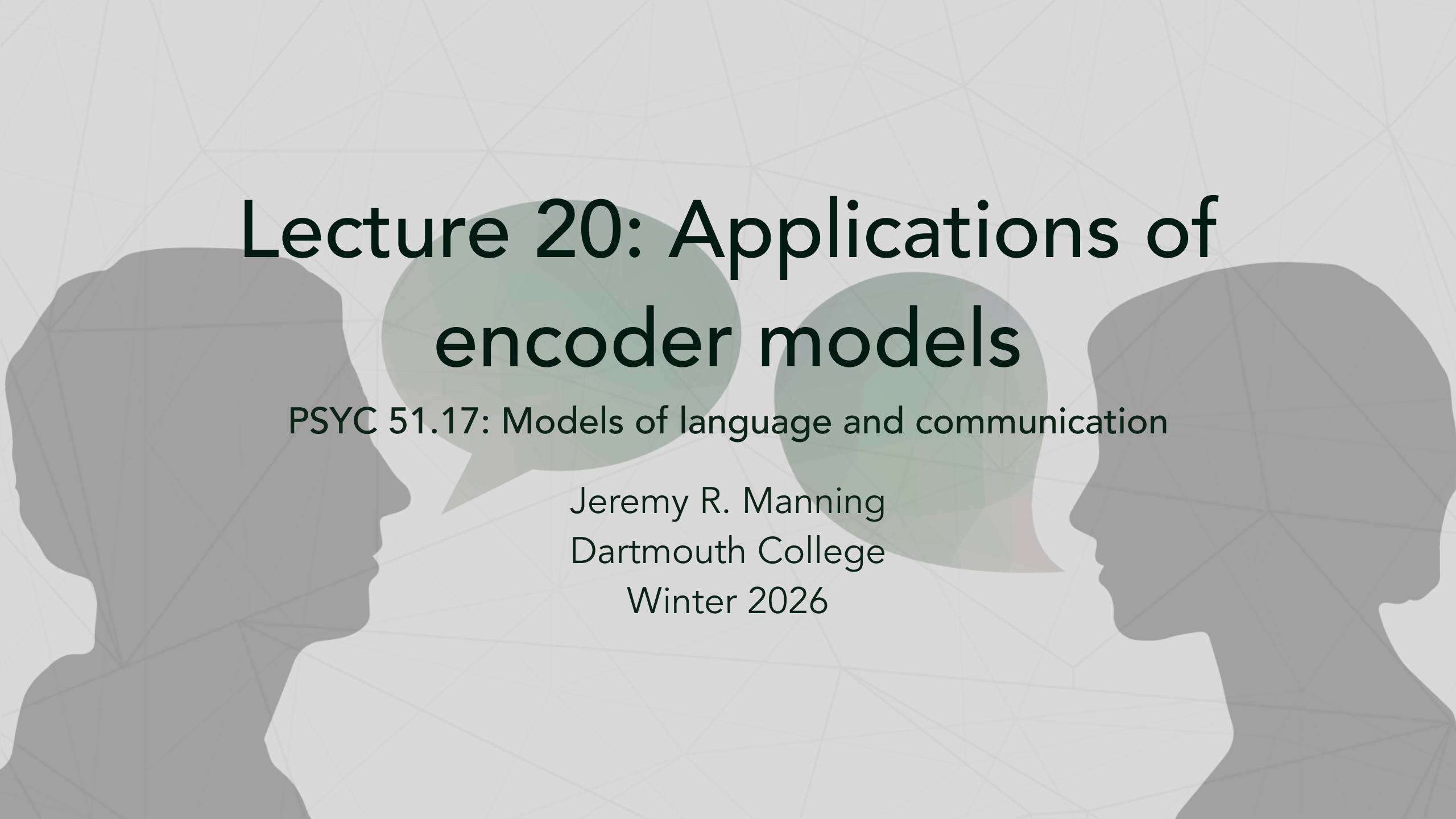




Lecture 20: Applications of encoder models

PSYC 51.17: Models of language and communication

Jeremy R. Manning
Dartmouth College
Winter 2026

Learning objectives

By the end of this lecture, you will be able to...

1. Apply BERT to real-world NLP tasks: classification, NER, QA, and semantic similarity
2. Explain how BERT improved Google Search and reshaped the NLP industry
3. Connect encoder representations to brain-imaging findings (fMRI encoding)
4. Evaluate whether language models "understand" or merely pattern-match
5. Describe systematic approaches to measuring bias in language models

Recall and roadmap

From earlier lectures

We introduced BERT's architecture in Lecture 12, explored what BERT actually learns in Lecture 18, and compared BERT variants (RoBERTa, ALBERT, DistilBERT, ELECTRA) in Lecture 19.

Today's focus

Today: how encoder models are *used* — in industry, in neuroscience, and in the ongoing debate about whether any of this counts as "understanding."

Companion notebook



Companion Notebook — hands-on with classification, NER, QA, and sentence similarity

Where BERT excels

BERT powers a wide range of understanding tasks

Classification tasks: Sentiment analysis, topic classification, spam detection, intent recognition

Token-level tasks: Named entity recognition (NER), part-of-speech tagging, word sense disambiguation

Span-level tasks: Question answering, extractive summarization, information extraction

Sentence-pair tasks: Semantic similarity, natural language inference, paraphrase detection

Industry impact

BERT powers Google Search (understanding queries), customer service chatbots, content moderation systems, and document understanding pipelines at companies worldwide.

Case study: Google Search

BERT revolutionized search in 2019

Before BERT, search engines primarily matched keywords. BERT enabled Google to understand *how words relate to each other* — especially prepositions, negations, and context words.

Why word order matters

```
1 query = "2019 brazil traveler to usa need a visa"
2
3 # Before BERT (keyword matching):
4 # Matches both "US traveler to Brazil" AND "Brazil traveler
  to US"
5
6 # With BERT (contextual understanding):
7 # Understands "to USA" means the traveler's DESTINATION is
  the US
8 # Correctly ranks: "Brazil citizen visa requirements for
  USA"
```

Question answering with BERT

Extractive QA: find the answer span in a passage

```
1 from transformers import pipeline
2
3 qa = pipeline("question-answering",
4               model="bert-large-uncased-whole-word-masking-
   finetuned-squad")
5
6 result = qa(
7     question="In what country is Normandy located?",
8     context="The Normans were the people who in the 10th and
   11th centuries "
9           "gave their name to Normandy, a region in France."
10 )
11 print(result)
12 # {'answer': 'France', 'score': 0.987, 'start': 134, 'end': 140}
```

How it works

BERT encodes the question and the passage into vectors, and then finds the best matching span in the passage.

Named entity recognition

Token-level classification with BERT

```
1 from transformers import pipeline
2
3 ner = pipeline("ner", model="dslim/bert-base-NER")
4
5 text = "Apple Inc. is headquartered in Cupertino, California."
6 entities = ner(text)
7
8 for e in entities:
9     print(f"{e['word']}: {e['entity']} (score:
10         {e['score']:.2f})")
11 # Apple: B-ORG (0.99)
12 # Inc: I-ORG (0.99)
13 # Cupertino: B-LOC (0.99)
14 # California: B-LOC (0.99)
```

Semantic similarity

Measuring sentence similarity with BERT embeddings

```
1 from transformers import BertTokenizer, BertModel
2 import torch.nn.functional as F
3
4 tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
5 model = BertModel.from_pretrained('bert-base-uncased')
6
7 def get_sentence_embedding(sentence):
8     inputs = tokenizer(sentence, return_tensors='pt',
9 padding=True, truncation=True)
9     outputs = model(**inputs)
10    return outputs.last_hidden_state[:, 0, :] # [CLS] token
11
12 sent1 = "The cat is sleeping on the couch"
```

continued...

Semantic similarity

Measuring sentence similarity with BERT embeddings

```
13 sent2 = "A feline is resting on the sofa"      # Paraphrase
14 sent3 = "The weather is nice today"           # Unrelated
15
16 emb1, emb2, emb3 = [get_sentence_embedding(s) for s in [sent1,
17 sent2, sent3]]
18
18 print(f"Paraphrase similarity: {F.cosine_similarity(emb1,
19 emb2).item():.3f}") # High
19 print(f"Unrelated similarity: {F.cosine_similarity(emb1,
emb3).item():.3f}") # Low
```

...continued

Encoders in industry (2025)

Encoder models are everywhere — even if you don't see them

The NLP market reached **\$7.73 billion** in 2025. Most of that is powered by encoder models, not chatbots:

Application	Scale	Architecture
Google Search	8.5B queries/day	BERT → MUM
Sentence-BERT	700K+ downloads/day	Bi-encoder
Microsoft Bing	Billions of queries	DeBERTa-based
E5 / NV-Embed	Leading retrieval benchmarks	Encoder embeddings
Content moderation	Billions of posts/day	Fine-tuned RoBERTa

Why not just use GPT-4?

GPT-4 costs ~\$30/M tokens and takes ~500ms per request. A fine-tuned DistilBERT handles classification at ~\$0.01/M tokens and ~5ms per request. For high-volume, single-task workloads, encoders are **3,000× cheaper** and **100× faster**.

Neural encoding with language models

Can we predict brain activity from BERT representations?

```
1 # Neural encoding experiment workflow
2 import numpy as np
3 from transformers import BertModel
4
5 # 1. Participant reads sentences while in fMRI scanner
6 sentences = ["The dog chased the cat", "She opened the door",
7             ... ]
8 brain_activity = fmri_scanner.record(sentences) # (n_sentences,
9                                                  n_voxels)
10
11 # 2. Extract BERT representations for same sentences
12 bert = BertModel.from_pretrained("bert-base-uncased")
```

continued...

Neural encoding with language models

Can we predict brain activity from BERT representations?

```
11 bert_embeddings = []
12 for sent in sentences:
13     outputs = bert(tokenizer(sent, return_tensors="pt"))
14     bert_embeddings.append(outputs.hidden_states[8].mean(dim=1)) #
    Layer 8
15
16 # 3. Train encoding model: BERT representations → brain activity
17 from sklearn.linear_model import Ridge
18 encoder = Ridge().fit(bert_embeddings[:80], brain_activity[:80])
19
20 # 4. Predict brain activity for held-out sentences
```

...continued...

Neural encoding with language models

Can we predict brain activity from BERT representations?

```
21 predictions = encoder.predict(bert_embeddings[80:])
22 correlation = np.corrcoef(predictions.flat,
23   brain_activity[80:].flat)[0,1]
   # Correlation ~ 0.3–0.5 in language areas (statistically
   significant!)
```

...continued

Key finding: BERT layer 8 best predicts semantic brain areas; layers 2–4 predict phonological areas.

Brain-LLM alignment frontiers

The relationship between LLMs and brains is deeper than expected

Caucheteux & King (2022): LLM activations predict brain activity in a **layer-specific** way — early model layers map to auditory cortex, deeper layers to high-level language areas. Published in *Communications Biology*.

Gao et al. (2025, *Nature Computational Science*): "Brain-like" artificial neurons emerge in LLMs trained only on text — units that encode syntax, semantics, and even spatial concepts, mirroring brain organization.

Mind's Transformer (Aw et al., 2026, ICLR)

The Mind's Transformer: Proposes that the brain implements a **Transformer-like** algorithm — not just analogous representations, but analogous *computation*. Attention-like gating and residual-stream-like updates appear across cortical circuits.

Why this matters: If the brain and LLMs converge on similar computational strategies, it suggests these are *good solutions* for processing sequential, context-dependent information — not just engineering coincidences.

Understanding vs pattern matching

The core philosophical question

Evidence FOR understanding:

- Captures syntax and semantics automatically
- Resolves lexical ambiguity based on context
- Generalizes to unseen examples
- Predicts human brain activity patterns
- Layer structure mirrors human language processing hierarchy

Evidence AGAINST understanding:

- No grounding in the physical world
- No sensory or social experience
- Brittle to adversarial examples
- No causal reasoning
- Might be "just" sophisticated pattern matching

The key question: Is there a meaningful difference between "understanding" and

Adversarial examples and brittleness

BERT can be fooled by simple tricks

```
1 from transformers import pipeline
2 classifier = pipeline("sentiment-analysis")
3
4 # Works correctly
5 classifier("This movie was absolutely wonderful!")
6 # → POSITIVE (0.9998)
7
8 # Adding irrelevant negative words flips the prediction!
9 classifier("This movie was absolutely wonderful! [SEP] bad bad
10 bad bad")
11 # → NEGATIVE (0.9234) # WRONG!
12
13 # Synonym substitution changes confidence
14 classifier("The food was good") # → POSITIVE (0.99)
15 classifier("The food was fine") # → POSITIVE (0.72) # Less
16 confident
```

Measuring bias systematically

Moving beyond anecdotes to systematic measurement

Recall from Lecture 11 that word embeddings encode societal biases (e.g., "doctor" closer to "man," "nurse" closer to "woman"). BERT inherits these same biases from its training data. But how do we measure bias *systematically* across thousands of contexts?

SAGED: systematic bias evaluation (Jiang et al., COLING 2025)

SAGED proposes a five-step pipeline for measuring bias in language models:

1. **S**elect social groups (e.g., gender, race, age)
2. **A**ssemble evaluation contexts (template sentences)
3. **G**enerate model outputs for each group × context
4. **E**valuate using statistical tests (not cherry-picked examples)
5. **D**ocument findings with effect sizes and confidence intervals

Key insight: Individual examples (like doctor/nurse) are compelling but misleading. Systematic evaluation across thousands of contexts reveals that biases are *real but*

Limitations of current encoder models

Despite impressive performance, significant limitations remain

Quadratic complexity: Self-attention scales as $O(n^2)$ with sequence length, limiting context to 512–4096 tokens. (ModernBERT pushes this to 8,192 with Flash Attention.)

No generation capability: BERT is designed for understanding, not producing text. For generation tasks, use decoder models (GPT) or encoder-decoder models (T5, BART).

Data hunger: Pre-training requires billions of words. Humans learn language with orders of magnitude less data.

No world model: BERT learns statistical patterns in text but has no grounding in physical reality, sensory experience, or causal reasoning.

Encoder vs decoder: the 2026 landscape

The architecture divide is narrowing

	Encoder (BERT)	Decoder (GPT)
Training	Masked language modeling	Autoregressive next-token
Context	Bidirectional	Left-to-right (causal)
Best for	Understanding tasks	Generation tasks
Cost	\$0.01/M tokens	\$1-30/M tokens
Latency	~5ms	~500ms

Where things are headed

Convergence: Decoder-only models (GPT-4, Claude) now do classification via prompting. Meanwhile, encoder-decoder hybrids (T5, UL2) blur the distinction further. The field is converging toward flexible architectures that can handle both understanding and generation — but cost and speed still favor purpose-built encoders for production.

Discussion

Questions to consider

1. **Understanding vs pattern matching:** If a model predicts brain activity, resolves ambiguity, and handles novel inputs — but has no physical experience — does it "understand"? What evidence would change your mind?
2. **The measurement problem:** SAGED shows bias is more nuanced than individual examples suggest. Does this make bias *less* concerning, or *more* concerning (because it's harder to detect)?
3. **Brain-model convergence:** If brains and LLMs independently converge on similar representations and computations, what does that tell us about the nature of language processing? Is there only one way to solve it?
4. **The cost-quality tradeoff:** GPT-4 can do anything an encoder can

Further reading

Further reading

Devlin et al. (2019, *NAACL*) "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding" — The original BERT paper.

Caucheteux & King (2022, *Communications Biology*) "Brains and algorithms partially converge in natural language processing" — Layer-specific brain-LLM alignment.

Aw et al. (2026, *ICLR*) "The Mind's Transformer" — Evidence that the brain implements Transformer-like computation.

Gao et al. (2025, *Nature Computational Science*) "Brain-like artificial neurons in LLMs" — Emergent brain-like units in language models.

Jiang et al. (2025, *COLING*) "SAGED" — Systematic bias evaluation pipeline for language models.

Bender & Koller (2020, *ACL*) "Climbing towards NLU" — The "understanding" debate.

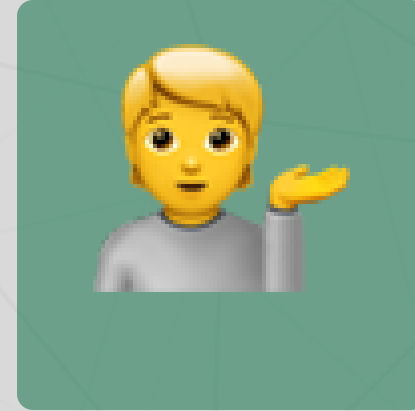
Questions?



Email



Discord



Office hours

Up next...

Week 7: GPT and autoregressive generation — from decoder architecture to scaling laws